

Lab 7 Placeholder

This lab covers CPU scheduling concepts including long-term, medium-term, and short-term schedulers. I will implement First-Come-First-Serve (FCFS) and Round-Robin scheduling algorithms in C#, then visualize the results using Gantt charts. The lab requires Visual Studio Code with C# and Project (available only on campus CS/CSE machines). All code must be properly commented, inputs validated, and meaningful variable names used. For each code we represent a flowchart and the related Gantt charts.

Long-Term Scheduling

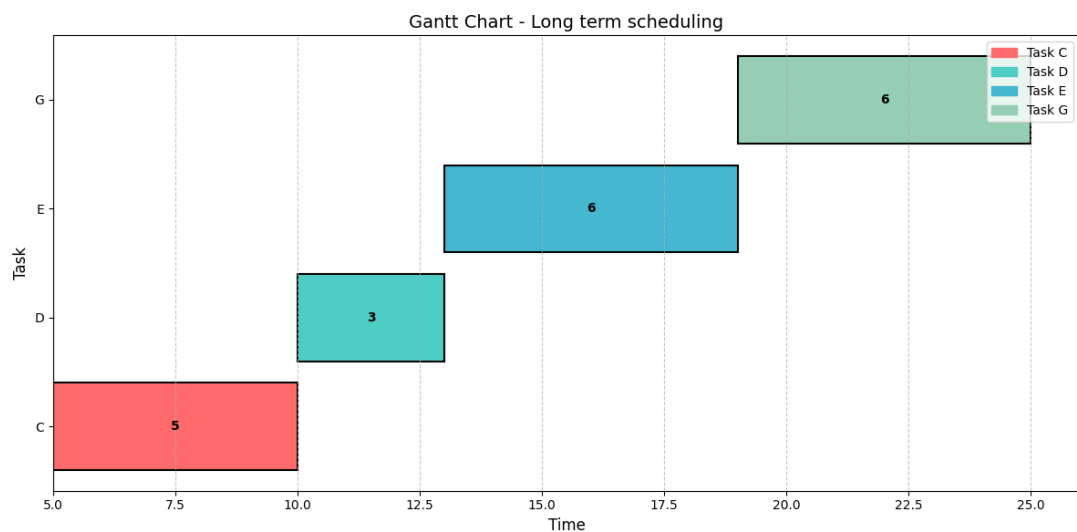
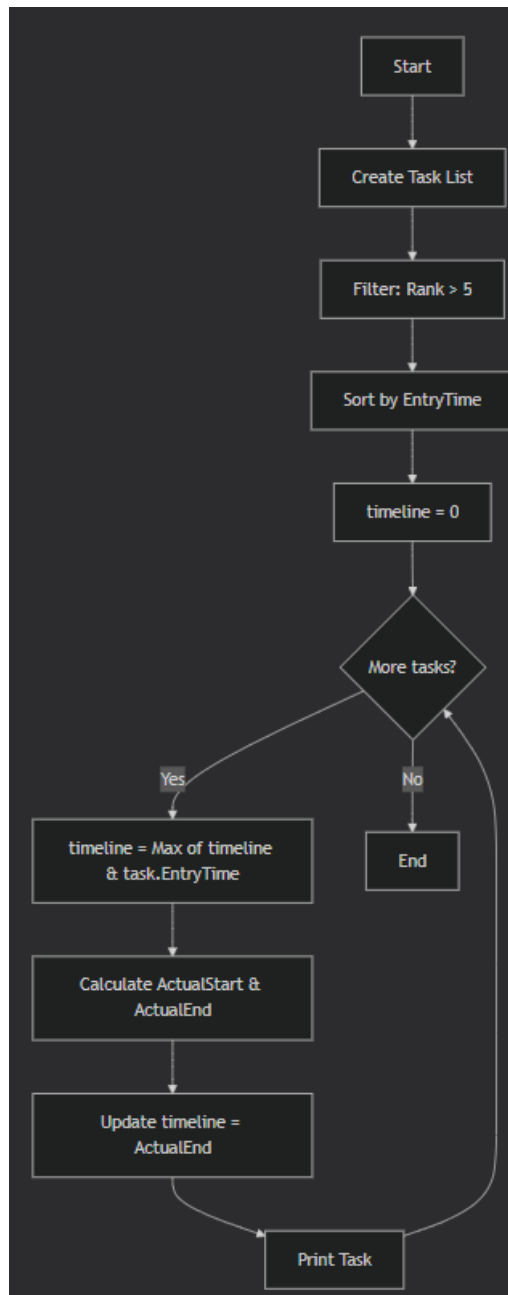
This lab covers CPU scheduling concepts including long-term, medium-term, and short-term schedulers. I implemented First-Come-First-Serve (FCFS) scheduling using the task data from **Table 1** (Tasks A–G with their Entry Time, Duration, and Priority/Rank). The code filters tasks where Rank > 5, sorts them by arrival time, then simulates FCFS execution tracking start and finish times. Gantt chart visualisation is included as required. All inputs are validated and variable names are meaningful.

Filtered Task Schedule:

```
[Task C] Priority: 8 | Arrived: 5 | Run: 5 | Started: 5 | Finished: 10
[Task D] Priority: 6 | Arrived: 6 | Run: 3 | Started: 10 | Finished: 13
[Task E] Priority: 10 | Arrived: 8 | Run: 6 | Started: 13 | Finished: 19
[Task G] Priority: 7 | Arrived: 10 | Run: 6 | Started: 19 | Finished: 25
```

Process	Arrival Time	Execution Time	Priority
A	0	3	5
B	2	6	4
C	5	5	8
D	6	3	6
E	8	6	10
F	9	2	3
G	10	6	7

Table 1: Long-term Scheduling Table



Short-Term Scheduling (CPU) Task 1: FCFS Scheduling

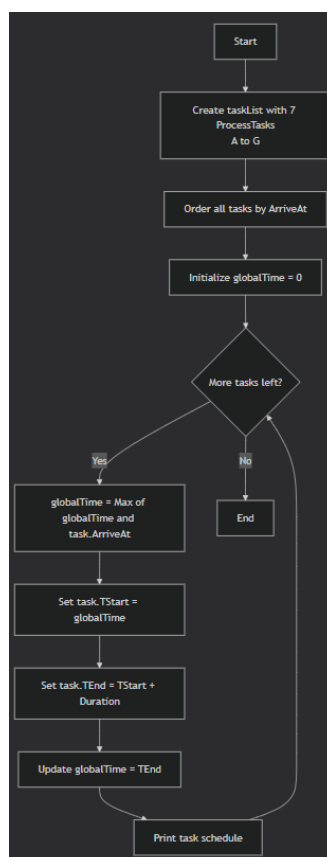
I implemented FCFS scheduling using Table 2 in C#. The program sorts processes by arrival time, then calculates for each process: Finish Time (Start + Execution), Turnaround Time (Finish - Arrival), and Normalized Turnaround Time (Turnaround / Execution). The Gantt chart visually represents the order of execution showing when each process runs on the CPU

--- Full Process Scheduling Report (A-G) ---

```
[ID: A] | Priority: 5 | Arrival: 0 | Exec: 3 | Window: 0 - 3
[ID: B] | Priority: 4 | Arrival: 2 | Exec: 6 | Window: 3 - 9
[ID: C] | Priority: 8 | Arrival: 5 | Exec: 5 | Window: 9 - 14
[ID: D] | Priority: 6 | Arrival: 6 | Exec: 3 | Window: 14 - 17
[ID: E] | Priority: 10 | Arrival: 8 | Exec: 6 | Window: 17 - 23
[ID: F] | Priority: 3 | Arrival: 9 | Exec: 2 | Window: 23 - 25
[ID: G] | Priority: 7 | Arrival: 10 | Exec: 6 | Window: 25 - 31
```

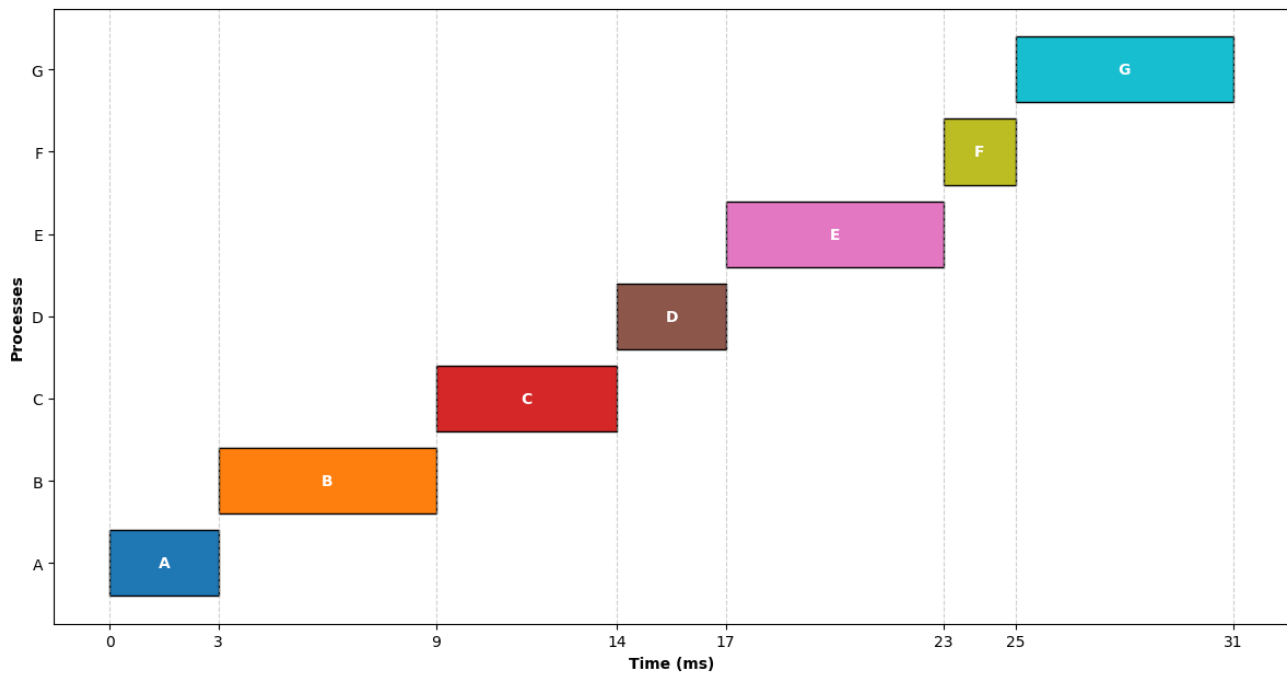
Process	Arrival Time	Execution Time
A	0	3
B	2	6
C	5	5
D	6	3
E	8	6
F	9	2
G	10	6

Table 2: FCFS process table



2. Short-Term Scheduling (CPU)

Task 1: FCFS Scheduling (Multi-Line Chart)



Round-Robin Scheduling

I implemented Round-Robin scheduling with time slices $TQ = 1, 3, 4$, and 6 , calculated finish, turnaround, and normalized turnaround times for each process, and plotted Gantt charts. Smaller time slices increased context switching and turnaround time; larger slices behaved like FCFS.

=====
Round Robin Scheduling with $TQ = 1$:
=====

Job A: Finish 4, Turnaround 4, Normalized 1.33
 Job D: Finish 18, Turnaround 12, Normalized 4.00
 Job F: Finish 19, Turnaround 10, Normalized 5.00
 Job B: Finish 22, Turnaround 20, Normalized 3.33
 Job C: Finish 24, Turnaround 19, Normalized 3.80
 Job E: Finish 30, Turnaround 22, Normalized 3.67
 Job G: Finish 31, Turnaround 21, Normalized 3.50

=====
Round Robin Scheduling with $TQ = 3$:
=====

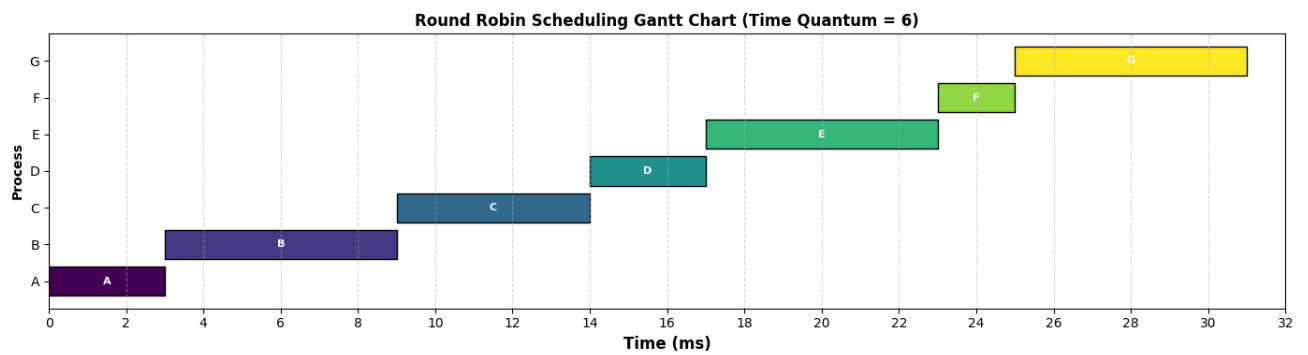
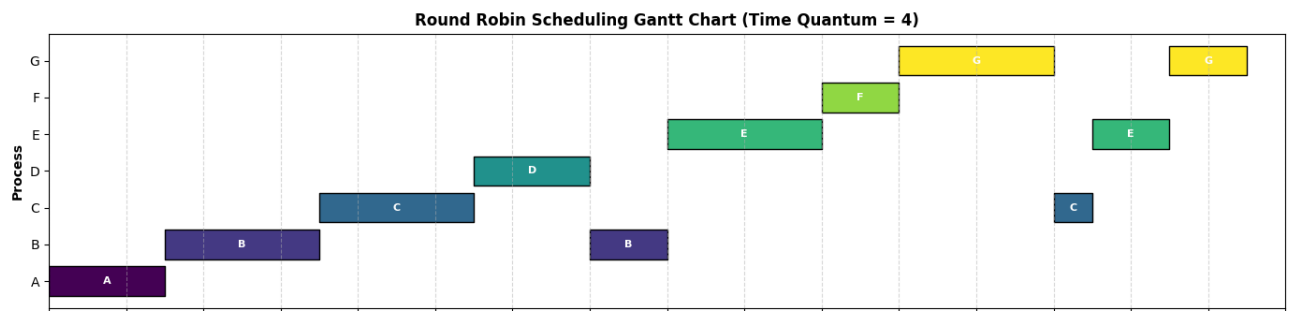
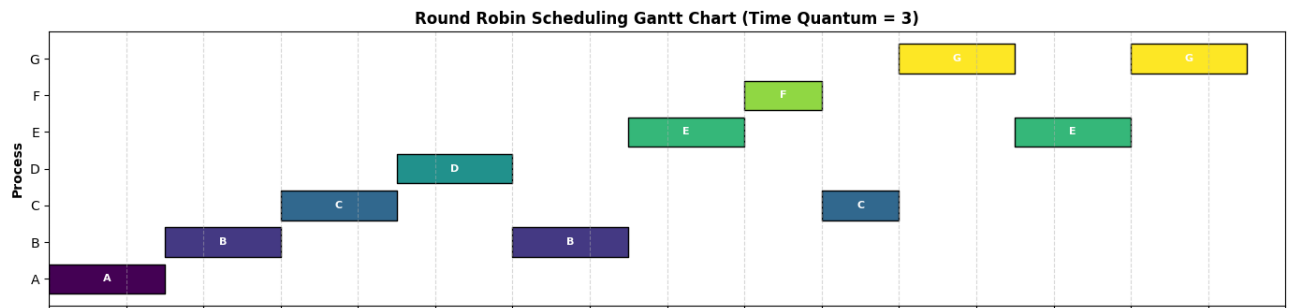
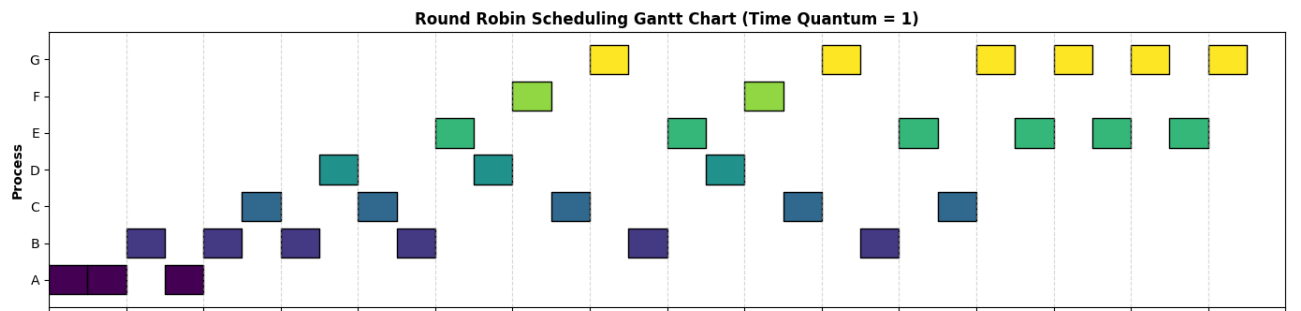
Job A: Finish 3, Turnaround 3, Normalized 1.00
 Job D: Finish 12, Turnaround 6, Normalized 2.00
 Job B: Finish 15, Turnaround 13, Normalized 2.17
 Job F: Finish 20, Turnaround 11, Normalized 5.50
 Job C: Finish 22, Turnaround 17, Normalized 3.40
 Job E: Finish 28, Turnaround 20, Normalized 3.33
 Job G: Finish 31, Turnaround 21, Normalized 3.50

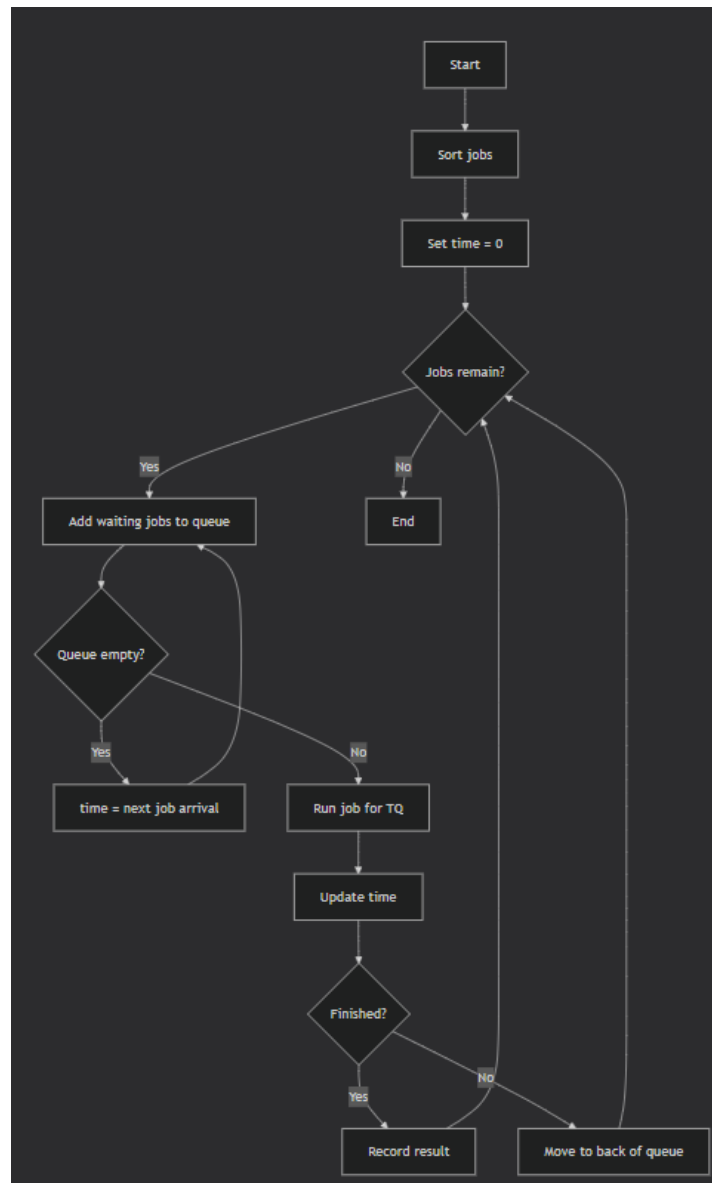
=====
Round Robin Scheduling with $TQ = 4$:
=====

Job A: Finish 3, Turnaround 3, Normalized 1.00
 Job D: Finish 14, Turnaround 8, Normalized 2.67
 Job B: Finish 16, Turnaround 14, Normalized 2.33
 Job F: Finish 22, Turnaround 13, Normalized 6.50
 Job C: Finish 27, Turnaround 22, Normalized 4.40
 Job E: Finish 29, Turnaround 21, Normalized 3.50
 Job G: Finish 31, Turnaround 21, Normalized 3.50

=====
Round Robin Scheduling with $TQ = 6$:
=====

Job A: Finish 3, Turnaround 3, Normalized 1.00
 Job B: Finish 9, Turnaround 7, Normalized 1.17
 Job C: Finish 14, Turnaround 9, Normalized 1.80
 Job D: Finish 17, Turnaround 11, Normalized 3.67
 Job E: Finish 23, Turnaround 15, Normalized 2.50
 Job F: Finish 25, Turnaround 16, Normalized 8.00
 Job G: Finish 31, Turnaround 21, Normalized 3.50





Context Switching

I extended the Round-Robin simulation to include priority-based scheduling using data from Table 2, with time quanta $TQ = 1$ and 6 . Priority determined execution order, while the time quantum controlled how long each process ran before context switching. The Gantt charts show that smaller time slices ($TQ = 1$) caused frequent context switches, increasing overhead, while larger slices ($TQ = 6$) reduced switching but allowed lower-priority processes to block higher-priority ones longer. Priority scheduling improved response time for high-priority tasks but increased waiting time for low-priority ones.

```

=====
ROUND ROBIN (TQ = 1)
=====
Timeline: [Start - End] Job (Remaining)
[ 0 - 1] Task A (Left: 2)
[ 1 - 2] Task A (Left: 1)
[ 2 - 3] Task B (Left: 5)
[ 3 - 4] Task A (Left: 0)
[ 4 - 5] Task B (Left: 4)
[ 5 - 6] Task C (Left: 4)
[ 6 - 7] Task B (Left: 3)
[ 7 - 8] Task D (Left: 2)
[ 8 - 9] Task C (Left: 3)
[ 9 - 10] Task B (Left: 2)
[10 - 11] Task E (Left: 5)
[11 - 12] Task D (Left: 1)
[12 - 13] Task F (Left: 1)
[13 - 14] Task C (Left: 2)
[14 - 15] Task G (Left: 5)
[15 - 16] Task B (Left: 1)
[16 - 17] Task E (Left: 4)
[17 - 18] Task D (Left: 0)
[18 - 19] Task F (Left: 0)
[19 - 20] Task C (Left: 1)
[20 - 21] Task G (Left: 4)
[21 - 22] Task B (Left: 0)
[22 - 23] Task E (Left: 3)
[23 - 24] Task C (Left: 0)
[24 - 25] Task G (Left: 3)
[25 - 26] Task E (Left: 2)
[26 - 27] Task G (Left: 2)
[27 - 28] Task E (Left: 1)
[28 - 29] Task G (Left: 1)
[29 - 30] Task E (Left: 0)
[30 - 31] Task G (Left: 0)

```

```

Final Metrics:
Job | Finish | Turnaround | Waiting
A   | 4       | 4           | 1
B   | 22      | 20          | 14
C   | 24      | 19          | 14
D   | 18      | 12          | 9
E   | 30      | 22          | 16
F   | 19      | 10          | 8
G   | 31      | 21          | 15

```

Average Waiting Time: 11.00 ms

```

=====
ROUND ROBIN (TQ = 6)
=====
Timeline: [Start - End] Job (Remaining)
[ 0 - 3] Task A (Left: 0)
[ 3 - 9] Task B (Left: 0)
[ 9 - 14] Task C (Left: 0)
[14 - 17] Task D (Left: 0)
[17 - 23] Task E (Left: 0)
[23 - 25] Task F (Left: 0)
[25 - 31] Task G (Left: 0)

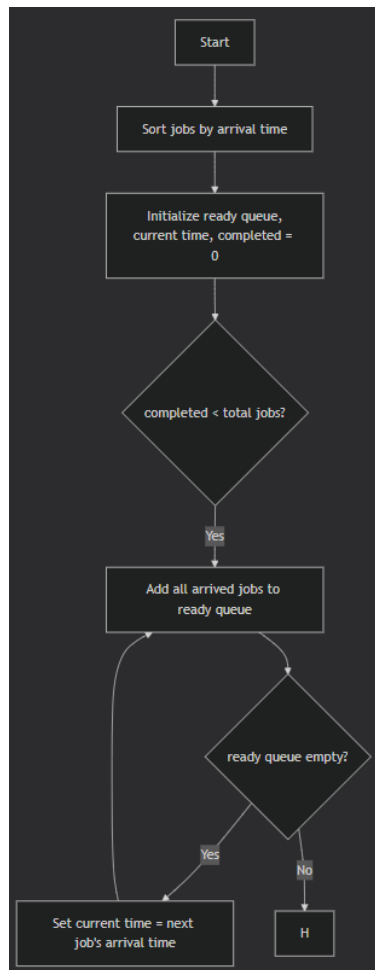
```

```

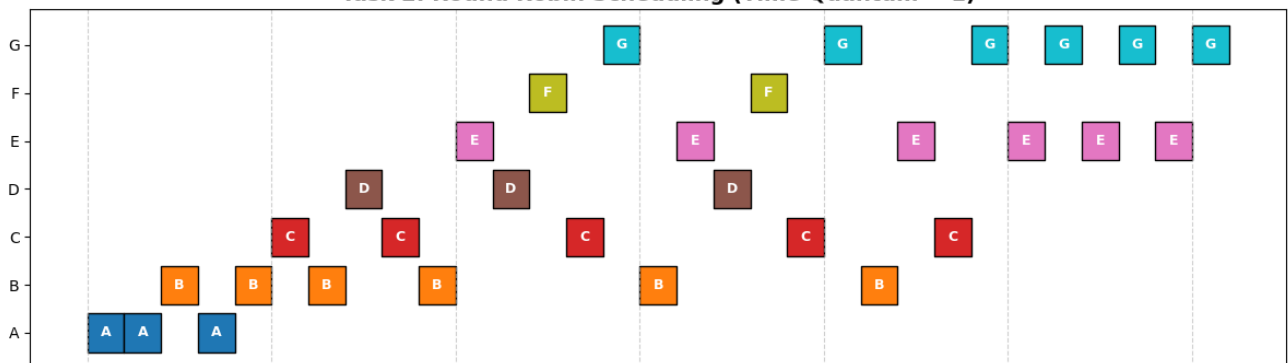
Final Metrics:
Job | Finish | Turnaround | Waiting
A   | 3       | 3           | 0
B   | 9       | 7           | 1
C   | 14      | 9           | 4
D   | 17      | 11          | 8
E   | 23      | 15          | 9
F   | 25      | 16          | 14
G   | 31      | 21          | 15

```

Average Waiting Time: 7.29 ms



Task 2: Round Robin Scheduling (Time Quantum = 1)



Task 2: Round Robin Scheduling (Time Quantum = 6)

